

1. Figure 9

Code:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/figure9_and_IoU_values_for_the_new_table3/fig9_saliency_map.py

Data:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/figure9_and_IoU_values_for_the_new_table3/biases&weights_optimized.pickle

Copied from:

“experimental_data\ex1\ex1_1_[784,48,35,10]+[784,50,10]_sigmoid_60,100\Trial_1”
which is inside the file https://github.com/gphehub/pnn2204/experimental_data.zip

Can be replaced by `biases&weights_optimized.pickle` from other trials of the sigmoid function (The code `fig9_saliency_map.py` is designed for the network using the sigmoid function. Therefore it will not generate the correct saliency map for the networks using the ReLU and Tanh functions without modification).

Instruction for running the code:

(1) Preparation:

Put the above `fig9_saliency_map.py`, `biases&weights_optimized.pickle`, and `mnist_loader.py` in your working directory;

Put the MNIST dataset file `mnist.pkl.gz` in the `/data` subdirectory;

(`mnist_loader.py` and `mnist_loader.py` are not included here. They are available at: <https://github.com/mnielsen/neural-networks-and-deep-learning/archive/master.zip>)

(2) Running the code:

Run `fig9_saliency_map.py`.

It will generate `he_fig9.png` and `disagreed_results.csv` in the working directory.

2. New Table 3: The IoU ratio

Code:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/figure9_and_IoU_values_for_the_new_table3/IoU.py

Data:

Input:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/figure9_and_IoU_values_for_the_new_table3/biases&weights_optimized.pickle

Output:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/figure9_and_IoU_values_for_the_new_table3/iou_complete_list.csv

Instruction for running the code:

Run IoU.py following the same instruction for running fig9_saliency_map.py, and it will generate iou_complete_list.csv in the working directory.

Then use the data in iou_complete_list.csv to compute the average IoU values shown in the new Table 3.

3. Table 4: Comparison between three activation functions

(The old Table 3 in previous versions of the manuscript)

Data:

The data for trials 10~23 is in the directory:

https://github.com/gphehub/pnn2204/additional_experiments_mar2026/table3/data

(Yes, due to poor network connection, I am unable to rename the directory “table3” as “table4”.)

The data for trials 1~9 can be obtained as follows:

Download and unzip https://github.com/gphehub/pnn2204/experimental_data.zip

The data for the sigmoid function is in the unzipped directory:

`experimental_data\ex1\ex1_4_[784,48,35,10]+[784,48,35,10]_sigmoid_0,100`

The data for the ReLU and Tanh functions is in the unzipped directory:

`experimental_data\ex4`

Code:

The codes with the parameters already set for computing the data for Table 4 are in the directory:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/table3/code

Instruction for running the code:

(1) Preparation:

Put the above codes, all the codes in <https://github.com/gphehub/pnn2204/tree/main/code>, and `mnist_loader.py` in your working directory;

Put the MNIST dataset file `mnist.pkl.gz` in the `/data` subdirectory;

(`mnist_loader.py` and `mnist_loader.py` are not included here. They are available at:

<https://github.com/mnielsen/neural-networks-and-deep-learning/archive/master.zip>)

(2) To compute new data for Table 4:

Run any of the following program:

`main2204_2Para6S-Table3.py` (for sigmoid)

`main2204_2Para6R-Table3.py` (for ReLU)

`main2204_2Para6T-Table3.py` (for Tanh)

They will create the following output files in the working directory:

`biases&weights_initialized.pickle`

`biases&weights_optimized.pickle`

`classification_accuracy.csv`

`disagreed_results.csv`

network2Para6S.pkl (or network2Para6R.pkl, or network2Para6T.pkl)

Open disagreed_results.csv, and use the data “Number of results when both subnetworks are wrong but the combined network is right” and “Para” as n_{IV} and α , respectively, for Table 4. The r_{IV} value in Table 4 is calculated as $(n_{IV}/10000)/\alpha$.

(3) To verify the existing data in Table 4:

Due to the randomness in the shuffling of the training data, loading biases&weights_initialized.pickle into the network and re-run the training process generally will *not* result in exactly the same existing data in Table 4.

Instead, to reproduce the existing data in Table 4, we should load biases&weights_optimized.pickle into the network using main2204_2Para_load_6R.py for ReLU (or main2204_2Para_load_6S.py for sigmoid, or main2204_2Para_load_6T.py for Tanh), and compute the classification result directly without re-training the network.

4. Table B.1

Data:

The data for Table B.1 is in the directory:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/tableB1/data

Code:

The codes for computing the data for Table B.1 are in the directory:

https://github.com/gphehub/pnn2204/tree/main/additional_experiments_mar2026/tableB1/code

Instruction for running the code:

(1) Preparation:

Put the above codes in your working directory;

Put the CIFAR-10 dataset file `cifar-10-python.tar.gz` in the `/data` subdirectory;

(`cifar-10-python.tar.gz` is not included here. It is available at:

<http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>)

(2) To compute new data for Table B.1:

Run any of the following program:

`PCNN2603-Sigmoid-CIFAR.py` (for sigmoid)

`PCNN2603-ReLU-CIFAR.py` (for ReLU)

`PCNN2603-Tanh-CIFAR.py` (for Tanh)

They will create the following output files in the `/data` subdirectory:

`biases&weights_initialized.pickle`

`biases&weights_optimized.pickle`

`classification_accuracy.csv`

`disagreed_results.csv`

Open `disagreed_results.csv`, and use the data “Ratio of Type IV results to correct results” and “Accuracy on the validation data” as r_{IV} and α , respectively, for Table B.1.

(3) To verify the existing data in Table B.1:

Due to the randomness in the shuffling of the training data, loading `biases&weights_initialized.pickle` into the network and re-run the training process generally will *not* result in exactly the same existing data in Table B.1.

Instead, to reproduce the existing data in Table B.1, we should load `biases&weights_optimized.pickle` into the network using any of the above PY program, and compute the classification result directly without re-training the network.

The command for loading `biases&weights_optimized.pickle`:

```
with open('biases&weights_optimized.pickle', 'rb') as f:  
    init_params = pickle.load(f)
```